The two objects are created without arguments, which means they both have an initial internal value of the current system timestamp, which is easily accessed via the `->getTimestamp()` method. Remember that in the previous section we constructed a `DateTime` instance with a value `-$epoch = new DateTime('1970-01-01 00:00:00');` – here we've omitted the string. More to come on this in a moment. The `DateTime::modify()` method accepts a string argument (usually in the form `'(+|-)n (days|hours|minutes)'`) that adjusts its timestamp value. In this case, we've added one day (which means the timestamp increments by the exact number of seconds in a day, which is *86400*). `DateTime` simply adjusts its timestamp value; `DateTimeImmutable`, however, does not, but it will return a new instance with the adjusted value:

```
$dti = new DateTimeImmutable();
var_dump(spl_object_id($dti));
$dti2 = $dti->modify('+1 day');
var_dump(spl_object_id($dti2));
```

```
Output:
int(1)
int(2)
```

`spl_object_id()` is part of the `SPL` (Standard PHP Library) module and shows us that `$dti` and `$dti2` are references to different objects. `SPL` offers many object utility functions and OOP APIs; we used `spl_autoload_register()` in Chapter 6, "Object Fundamentals." The `->modify()` method, when called on a `DateTime` instance, returns a reference to the same object, rather than a new instance. Throughout the rest of this section, wherever the `DateTime` class is used, you can also substitute for `DateTimeImmutable`. They have exactly the same interface and behavior, with the caveat demonstrated above.

Building `DateTime` Objects

We've seen `DateTime` (and its immutable counterpart) constructed with and without arguments. The following demonstrates how flexible the constructor argument can be, any validly formatted date/time string can be used to instantiate a `DateTime` object, when we come to the `DateTime::format()` method, you'll see just how many ways there are to express dates and times. For now, here's a quick taster: